SIMATS ENGINEERING

CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**192424390**
Name	**Venu G**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Code of Conduct:

I _____Venu(192424390)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5
Decision Making	Makes well-reasoned, logical decisions	Makes reasonable decisions with	Makes weak decisions with	No clear decisions made or rationale provided	10

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
	with strong rationale	some justification	limited justification		
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50

Question 1 — Emergency Medicine Drone: Greedy Best-First Search vs. A*

i. Behaviour of Greedy Best-First Search

Greedy Best-First Search (GBFS) selects the next node to expand using only the heuristic value h(n) — here, straight-line distance to the delivery point — and completely ignores the actual cost incurred so far, g(n). In this scenario, the drone would always move toward whichever waypoint looks geographically closest to the destination, regardless of whether the terrain there is flooded, the weather has made that path dangerous, or a detour would actually be faster and safer.

Strengths: GBFS is fast and computationally cheap, since it only evaluates one number per node and does not need to track accumulated cost — this matches the drone's urgent, low-latency decision requirement well, and it can reach a destination quickly when the heuristic happens to align with the real terrain.

Weaknesses under uncertainty: because it ignores real movement cost, GBFS can be lured toward a path that looks short on a straight-line map but is actually blocked, flooded, or far more costly under current weather — it has no mechanism to "reconsider" a route once a more direct-looking option appears, so it can commit to unsafe or inefficient paths. It is also not optimal and not even guaranteed to avoid revisiting problematic areas, since newly blocked paths discovered mid-flight are not weighed against alternative routes already dismissed.

ii. Behaviour of A* Search

A* evaluates nodes using f(n) = g(n) + h(n), combining the actual accumulated travel cost so far (which can incorporate weather-adjusted and terrain-adjusted movement cost) with the heuristic estimate of remaining distance. This means the drone does not just chase whatever looks geographically nearest — it weighs how expensive/risky the path actually was to reach that point, so a seemingly short straight-line route through a high-cost flooded zone is properly penalized against a longer but cheaper, safer route.

- If the heuristic is admissible (never overestimates true remaining cost) and the search space is properly modeled, A* is guaranteed to find the minimum-cost (optimal) path, which in this scenario directly translates to minimizing real delivery delay and risk, not just geographic distance.
- The trade-off is computational: A* must maintain and compare cumulative costs across multiple candidate paths (using a priority queue over f(n)), which requires more memory and computation than GBFS's simpler heuristic-only comparison.

iii. Impact of Heuristic Accuracy on Both Algorithms

Aspect	Greedy Best-First Search	A* Search

Role of heuristic accuracy	The heuristic is the only factor guiding search — its accuracy is everything	The heuristic guides efficiency, but cost g(n) still anchors the decision
Effect of an inaccurate/misleading heuristic	Can lead the drone directly into blocked or hazardous zones with no correction	Search may explore more nodes (slower), but final path quality degrades only mildly if heuristic stays admissible
Effect of an inadmissible heuristic (overestimates cost)	Already non-optimal by design, so this makes a bad situation only slightly worse	Loses its optimality guarantee entirely — may return a costlier, more dangerous route while appearing confident
Effect of a highly accurate heuristic	Improves route quality but never yields a formal optimality guarantee	Search becomes extremely efficient, approaching a direct path to the true optimal route

iv. Effect of Dynamic Changes (Blocked Paths) on Search Performance

Both algorithms, in their classical form, assume the graph of available paths is static once search begins — but the flood scenario is inherently dynamic, with roads becoming inaccessible mid-mission.

- For GBFS, a sudden blockage simply forces it to re-evaluate based on heuristic distance again, which is cheap to recompute — but this repeated "chase the nearest-looking option" behaviour can cause oscillation or wasted backtracking, since it has no memory of cost already spent detouring.
- For A*, a blockage invalidates part of the cost model built so far, and a naive re-run of A* from scratch after every update becomes expensive in a time-critical mission. In practice, this is addressed with incremental / real-time variants such as D* or D* Lite, which reuse previous search effort and only recompute the portions of the path affected by the newly blocked route, keeping cost-awareness while staying responsive to real-time updates.
- In general, the more frequently the environment changes, the more a pure offline A* recomputation becomes a bottleneck — favoring either lightweight heuristic-only reactions (GBFS-like, at the cost of safety) or smarter incremental replanning that preserves A*'s cost-awareness without paying the full recomputation cost every time.

v. Recommendation and Justification

For this scenario, a cost-aware, incrementally-replanning approach — effectively an A* variant such as D* Lite or Anytime Weighted A* — is the most suitable choice, rather than pure Greedy Best-First Search:

- Safety and optimality matter more than raw speed here: an emergency-medicine delivery that takes a slightly longer but reliably passable route is far preferable to a

"fast" decision that leads the drone into a flooded or blocked area, which could cause total mission failure and delay far worse than a suboptimal detour.

- Real-time constraints can still be met by using an anytime or incremental variant of A* — it does not need to restart search from scratch after every update, and can return a "good enough" path immediately while continuing to refine it, giving the responsiveness of GBFS without abandoning cost-awareness.
- Pure GBFS remains attractive only if the mission genuinely cannot tolerate any computational overhead (e.g., extremely constrained onboard hardware) and some risk of a suboptimal or unsafe route is acceptable — which is not the case for emergency medical delivery.

Question 2 — Smart City Traffic Signal Optimization

Formulating the Problem as Optimization, and Why Traditional Search Is Inefficient

This problem is best formulated as an optimization problem rather than a classical path-finding search problem: the "state" is a complete configuration of signal timings (green/red durations and offsets) across every intersection, and the objective function is a weighted combination of total waiting time, fuel consumption, and congestion level, which the system tries to minimize. Unlike pathfinding, there is no meaningful "path" to a goal state that needs to be tracked — only the quality of the final (or continuously updated) configuration matters.

- The state space is combinatorially huge: with hundreds of intersections, each having multiple continuous or finely-discretized timing parameters, the number of possible configurations grows exponentially, making systematic/exhaustive search (BFS, DFS, even A*) computationally infeasible.
- There is no natural "goal test" in the classical search sense — the system is not searching for a single terminal state but continuously trying to improve a running configuration as traffic conditions evolve, which is exactly the setting local/optimization search methods are designed for, rather than classical goal-directed search.
- Because traffic patterns change dynamically, any solution computed offline for one traffic distribution can quickly become stale, so the chosen method must also be able to adapt cheaply rather than being recomputed from scratch.

i. Hill Climbing in This Scenario, and Its Limitations

Hill Climbing would start from an initial signal-timing configuration and repeatedly make small adjustments (e.g., extend or shorten a particular intersection's green phase), accepting a change only if it improves the objective function (lower total waiting time/fuel/congestion), and stopping when no single small change improves things further.

- Limitation — greedy and short-sighted: it only accepts strictly improving neighbor states, so it cannot make a temporarily "worse" change even if that change would enable a much better configuration a few steps later (e.g., adjusting two coordinated intersections together).
- Limitation — sensitive to starting point: the quality of the final configuration depends heavily on where the search starts, since it always converges to whatever peak is nearest, not necessarily the best one available.
- Limitation — poor fit for a constantly changing objective: because traffic flow changes continuously, whatever local optimum Hill Climbing settles into may become outdated almost immediately, and Hill Climbing itself is not designed for a moving target.

ii. Local Maxima, Plateaus, and Ridges — Real-World Interpretation

Issue	Real-World Meaning in Traffic Signal Optimization
Local maxima	A signal-timing configuration that cannot be improved by adjusting any single intersection alone, even though a genuinely better citywide configuration exists — e.g., a coordinated “green wave” along a corridor requires several intersections' offsets to change together, which single-step Hill Climbing cannot discover.
Plateaus	Regions where many nearby configurations produce almost identical waiting times — e.g., shifting a signal's green phase by one or two seconds makes no measurable difference — causing Hill Climbing to wander without clear improvement signals, potentially stopping prematurely.
Ridges	Situations where an improvement requires two or more changes made simultaneously (e.g., adjusting two adjacent intersections' offsets together to create a coordinated flow), but any single one of those changes in isolation looks neutral or even worse, so Hill Climbing — which only evaluates one change at a time — cannot find the direction of improvement.

iii. How Simulated Annealing or Genetic Algorithms Overcome These Limitations

Simulated Annealing: unlike Hill Climbing, it occasionally accepts a worse configuration, with the probability of accepting a worse move controlled by a “temperature” parameter that gradually decreases over time. Early in the search (high temperature), this lets the algorithm escape local maxima and cross plateaus/ridges by taking exploratory, temporarily suboptimal steps; as temperature cools, the search increasingly behaves like Hill Climbing, refining toward a strong final configuration. This makes it well suited to a search space riddled with local optima, like citywide signal timing.

Genetic Algorithms: instead of refining one configuration at a time, a GA maintains a whole population of candidate signal-timing configurations simultaneously, and evolves them through selection (favoring lower-cost configurations), crossover (combining parts of two good configurations, such as one intersection's timing from one “parent” and another's from a second “parent”), and mutation (random small tweaks). This parallel, population-based exploration naturally covers many regions of the huge search space at once, making it much less likely to get permanently trapped in any single local maximum, and it can be re-run or continuously evolved as traffic data updates, supporting adaptability.

iv. Recommended Approach and Justification

A Genetic Algorithm (potentially combined with elements of Simulated Annealing, or used to periodically re-optimize while a faster local method fine-tunes in between) is the recommended approach:

- Scalability: because a GA evolves a population of full candidate solutions in parallel, it scales more gracefully to the huge combinatorial space of hundreds of intersections than a single-path method like Hill Climbing, which must serially explore step by step.
- Adaptability: since traffic patterns evolve continuously, a GA's population can be continuously re-evaluated against updated real-time traffic data, allowing the system to gradually shift its “best known” configurations as conditions change, rather than needing a full restart.
- Avoiding entrapment: the combination of population diversity (GA) and/or controlled random exploration (Simulated Annealing) directly targets the local maxima, plateaus, and ridge problems that make Hill Climbing unreliable in this domain.
- In practice, a hybrid is often most effective: a GA (or SA) run periodically/offline-ish to find strong globally-coordinated timing plans, paired with a fast local Hill-Climbing-style adjustment for minor real-time fine-tuning between full re-optimizations — balancing solution quality with the need for a fast, continuously responsive system.

Question 3 — Autonomous Mars Rover: Online Search Agent

i. Why This Requires an Online Search Agent Instead of Offline Search

Offline search algorithms (like standard A* or BFS/UCS on a known graph) compute a complete solution path before any action is taken, which requires full knowledge of the state space in advance. The rover has no such complete map of Mars terrain — it only knows what its sensors can currently observe. This means the rover must act, observe the result, update its knowledge, and decide the next action — repeating this cycle continuously. This interleaving of computation and action, driven by an initially unknown environment, is precisely the definition of an online search agent, and is unavoidable here since a correct offline plan simply cannot be computed without the missing map information.

ii. Challenges of Partial Observability and Dynamic Environments

- Limited sensor range/field of view means the rover only perceives its immediate surroundings, so hazards just beyond current sensor range (a hidden crater behind a ridge, for instance) are invisible until the rover moves closer — this creates real risk of committing to an action that turns out to be unsafe only after execution.
- Some actions may be effectively irreversible (e.g., driving into loose sand or over unstable terrain that damages the rover), so the rover cannot always simply "backtrack" the way a search algorithm operating on an abstract graph can — this requires cautious, safety-aware action selection rather than pure cost-minimization.
- Communication delay with Earth (light-time delay of several minutes) means the rover largely cannot rely on real-time human guidance and must make most navigation decisions autonomously, which raises the stakes of correct in-the-moment decision-making.
- Dynamic surface conditions (dust storms, shifting sand, lighting changes affecting sensors) mean even the rover's own prior observations can become outdated, so its internal model must be continuously refreshed, not just built once.

iii. How the Rover Builds and Updates Its Internal Model

As a model-based agent, the rover maintains an internal representation of the terrain it has explored so far — typically an occupancy grid or map structure marking which cells are known-free, known-hazardous, or still unexplored. Using its sensors (cameras, LiDAR, or stereo vision), it incorporates each new observation into this map (a process similar to SLAM — Simultaneous Localization and Mapping), refining both its estimate of its own position and its knowledge of the surrounding terrain. This continuously updated internal model is what allows the rover to make locally-informed decisions (e.g., re-route around a newly detected rock) without needing global knowledge of the entire planet's terrain.

iv. Online Search vs. Uninformed and Informed (Offline) Search Strategies

Aspect	Uninformed/Informed Offline Search (BFS, UCS, A*)	Online Search (e.g., Online DFS, LRTA*)
Map knowledge required	Full state space/graph must be known before planning	Builds knowledge incrementally as the agent explores
When the plan is computed	Entirely before execution begins	Interleaved with execution — decide, act, observe, repeat
Behaviour under uncertainty	Not applicable / assumes a static, fully known environment	Explicitly designed to handle unknown or partially known environments
Optimality guarantee	A* guarantees optimal path given a complete, admissible-heuristic model	Generally weaker guarantees (bounded/competitive performance at best), since decisions are made with incomplete information
Suitability for the rover	Infeasible — no complete map exists in advance	Necessary and appropriate — matches the rover's actual information constraints

v. Justification of the Most Suitable Approach

The most suitable approach is a model-based online search agent using an informed, incremental algorithm — such as Learning Real-Time A* (LRTA*) or an incremental replanning method like D* Lite — rather than either a naive online algorithm with no heuristic guidance or any offline planning method:

- Uncertainty: since the rover cannot know the full terrain in advance, only an online approach that senses and updates its model as it moves can function correctly at all — ruling out offline search outright.
- Adaptability: an incremental, heuristic-guided online method allows the rover to reuse previously computed knowledge/paths rather than recomputing everything from scratch whenever a new hazard is detected, keeping the rover responsive as new terrain is revealed.
- Safety: combining a heuristic (informed) component with cautious, model-based updates lets the rover favor paths that are not just short but also avoid areas of uncertain or hazardous terrain, better matching the mission's priority on avoiding irreversible, damaging mistakes over pure speed.

Question 4 — University Exam Timetabling as a Constraint Satisfaction Problem

Formulating the Problem as a CSP

i. Variables, Domains, and Constraints

The timetabling problem maps naturally onto the standard CSP framework of variables, domains, and constraints:

Element	Definition in This Scenario
Variables	One variable per exam/subject to be scheduled (e.g., $Exam_1$, $Exam_2$, …, Exam□) — each variable represents "when and where this exam takes place."
Domains	For each exam variable, the domain is the set of valid (time slot, exam hall) pairs it could be assigned — restricted from the start by hall capacity versus expected student count, and by any special-accommodation requirements (e.g., only accessible rooms for certain students).
Constraints — No overlapping exams	Binary constraint between any two exam variables that share at least one student: their assigned time slots must differ.
Constraints — Limited exam halls	For any given time slot, no more exam variables can be assigned to the same hall simultaneously than the hall allows (effectively an all-different style constraint per slot/hall combination), and each hall's capacity must accommodate the assigned exam's student count.
Constraints — Subject precedence	Ordering constraints between specific pairs of exam variables (e.g., Exam A's time slot must be strictly earlier than Exam B's), reflecting curriculum dependencies.
Constraints — Faculty availability	Unary/binary constraints restricting an exam's assigned time slot to fall only within time slots when its required invigilating/faculty staff are available.
Constraints — Special accommodations	Unary constraints on specific exam variables restricting their domain to particular rooms (e.g., accessible rooms) or requiring extra time, which may also affect that exam's allowed slot length.

ii. How Backtracking Search Works in This Scenario

Backtracking search assigns a value (a specific time-slot/hall pair) to one exam variable at a time, checking after each assignment whether all constraints involving already-assigned variables are still satisfied.

- If an assignment violates a constraint (e.g., it overlaps another exam sharing a student, or exceeds the room's capacity), the search backtracks — undoes the assignment and tries a different value from that variable's domain.
- If no valid value remains for a variable, the search backtracks further, undoing a previous variable's assignment and trying an alternative there instead.
- This continues until either every exam variable has a consistent assignment (a complete, valid timetable) or the search exhausts all possibilities (indicating the constraints as given cannot all be satisfied). Variable- and value-ordering heuristics (e.g., choosing the most constrained exam first, or trying the least-constraining slot first) are typically used to make this process more efficient in practice.

iii. How Constraint Propagation (e.g., Forward Checking) Improves Efficiency

Rather than only checking constraints after a full assignment is made, constraint propagation techniques proactively reduce the domains of not-yet-assigned variables as soon as a related variable is assigned.

- Forward checking: whenever an exam is assigned a time slot, it immediately removes that time slot from the domains of all other exams that share a student with it (or otherwise conflict), rather than waiting to discover the conflict later through a failed assignment.
- This can detect a dead end much earlier — if forward checking empties another exam variable's domain completely, the search can backtrack immediately, without wasting time exploring further doomed assignments down that branch.
- More advanced propagation (e.g., full arc consistency via AC-3) goes further, propagating these domain reductions across chains of related variables, shrinking the effective search space substantially before backtracking even needs to occur, which is especially valuable given how large and densely interconnected a university-wide exam schedule's constraint graph can be.

iv. Real-World Challenges and the Best Strategy for Scalability

- Scale: with a large university, the number of exam variables and the density of shared-student overlap constraints grow quickly, making pure backtracking (even with propagation) potentially slow, since exam scheduling is, in the worst case, an NP-complete class of problem.
- Mixed hard and soft constraints: some requirements (e.g., no overlapping exams) are truly hard constraints that must never be violated, while others (e.g., student/faculty preferences for particular time slots) may be soft — a real system needs to distinguish between constraints that can never be broken and those that can be minimized rather than strictly enforced.

- Dynamic changes: late course additions, room unavailability, or last-minute faculty conflicts require the system to adjust an already-generated timetable efficiently, rather than recomputing everything from scratch.

For scalability, the recommended strategy combines: (1) strong constraint propagation (forward checking or arc consistency) to prune the search space early, (2) intelligent variable/value ordering heuristics (most-constrained-variable, least-constraining-value) to guide backtracking efficiently, (3) intelligent backtracking methods such as backjumping to avoid redundantly re-exploring the same failure repeatedly, and (4) for very large or frequently changing instances, a local-search repair method such as min-conflicts (starting from a complete but possibly constraint-violating assignment and iteratively fixing the most-conflicted exam) to quickly patch a timetable after small real-world changes, rather than re-solving the whole CSP from scratch each time.

Question 5 — Strategic Game AI: Minimax and Alpha-Beta Pruning

i. How the Minimax Algorithm Models This Problem

Minimax treats the game as a tree of alternating decision layers: MAX nodes represent the AI's own turn (choosing the move that maximizes its eventual outcome) and MIN nodes represent the opponent's turn (assumed to choose the move that minimizes the AI's outcome, i.e., playing optimally against the AI). Values are propagated upward from terminal (or heuristically evaluated) states back to the root, and at each MAX node the AI selects the action leading to the highest guaranteed value, assuming the opponent will always respond with their best possible counter-move. This models the adversarial, worst-case nature of competing against a skilled human player.

ii. Computational Challenges of Large Game Trees

- Real-time strategy games have an enormous branching factor: many units can each take multiple possible actions per turn, and these combine combinatorially, unlike simpler board games such as chess or tic-tac-toe.
- The effective depth needed to make good decisions can also be large, since RTS strategies often require reasoning many moves ahead (build orders, troop positioning, resource allocation) — so total nodes to evaluate grow as roughly b^d, which becomes computationally infeasible to explore fully within the strict time limits an RTS game demands.
- Full game trees essentially never reach terminal (win/lose) states within a practical search horizon, so the search must be depth-limited and rely on a heuristic evaluation function partway down the tree rather than exhaustively searching to the end of the game.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning augments Minimax by tracking two bounds while traversing the tree: alpha (the best value the MAX player can guarantee so far) and beta (the best value the MIN player can guarantee so far).

- Whenever, during the search, a node's value is found to be worse for the current player than a value already guaranteed elsewhere in the tree (i.e., beta ≤ alpha), the remaining branches under that node can be safely skipped — they cannot possibly change the final decision at the root, since a rational opponent would never allow the game to reach that branch.
- Crucially, this pruning does not sacrifice any accuracy: Alpha-Beta produces exactly the same decision that full Minimax would, just without wasting time evaluating branches that provably cannot affect the outcome.

- With ideal move ordering (evaluating the most promising moves first), Alpha-Beta pruning can reduce the effective branching factor from b to roughly √b, lowering the time complexity from O(b^d) to O(b^(d/2)) in the best case — allowing the search to look roughly twice as deep in the same time budget, which is highly valuable for an RTS AI needing both speed and strategic depth.

iv. Designing an Evaluation Function

Since the search must be cut off well before terminal game states in an RTS setting, a heuristic evaluation function is needed to estimate how favorable a given (non-terminal) game state is for the AI. Relevant factors to combine (typically as a weighted sum) include:

- Military strength: number, type, and health of units on each side, weighted by unit combat effectiveness.
- Economic strength: resources gathered/available, production capacity, and economic infrastructure, since this predicts future military strength.
- Map control/territory: area controlled, key strategic positions held (chokepoints, resource nodes), and visibility/scouting coverage.
- Positional and tactical factors: unit positioning advantages (e.g., flanking opportunities, high ground), vulnerability to counter-attack.
- Time-sensitive factors: upcoming reinforcements, technology/upgrade timers, and momentum (whether a side's position is improving or declining).

These factors are combined into a single weighted score, with weights typically tuned through playtesting, self-play, or learning-based methods, so that the evaluation function serves as a reliable proxy for “likelihood of eventually winning” without needing to search all the way to an actual game-ending state.

v. Recommended Strategy for Balancing Optimality and Real-Time Performance

- Iterative deepening with Alpha-Beta pruning and a time cutoff: the AI searches to depth 1, then 2, then 3, and so on, always keeping the best move found from the last fully completed depth — this guarantees a valid move is always ready the instant the time limit is reached, giving “anytime” behaviour appropriate for strict real-time constraints.
- Strong move ordering (e.g., trying previously good moves first, using killer-move and history heuristics) to maximize the benefit of Alpha-Beta pruning and reach greater effective depth within the time budget.
- Transposition tables to cache and reuse evaluations of previously seen game states reached via different move orders, avoiding redundant computation in a game with many transpositions.
- For extremely large branching factors where even pruned Minimax struggles (as is common in full-scale RTS games), a complementary or alternative approach such as

Monte Carlo Tree Search (MCTS) can be considered, since it handles large action spaces by sampling promising lines rather than exhaustively branching — but for the moderate, well-structured decision points within an RTS turn, iterative-deepening Alpha-Beta with a well-tuned evaluation function remains a strong, well-understood baseline that reliably balances move quality (optimality under the model) against the strict real-time response the game requires.